Walchand College of Engineering, Sangli

7CS353 Machine Learning Lab — Assignment 3

Data Normalisation: Five Techniques and Their Visual Effects

Name - Atharv Vinayak Kulkarni

PRN - 245109074

Class - TY CSE(B)

Solutions for All Tasks from Assignment 3

B.1 Generate and Explore the Raw Data

B1. Load, Inspect and Compare Mean vs Median

Task: Load normalisation_data.csv. Print shape, head, and describe(). In a markdown cell, record for each column: the mean, median, min, max, and whether mean is above or below median. Explain what a large gap between mean and median tells you about shape.

Solution:

```
[2]: df=pd.read_csv("normalisation_data.csv")
     print(df.shape)
     print(df.head())
     df.describe().round(2)
```

```
(200, 2)
      bp  wait
0  124.6  64.2
1  104.4  53.6
2  131.3   5.2
3  134.1  23.9
4   90.7  23.1
```

[2]:

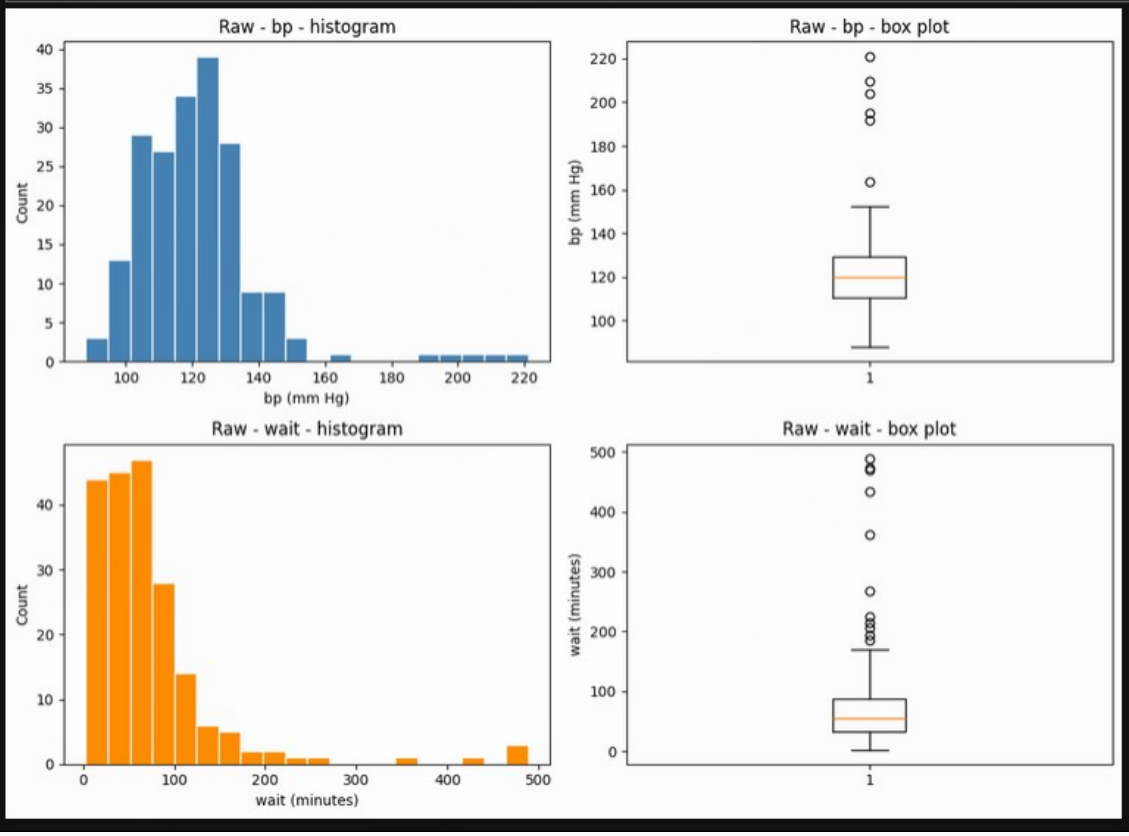
	bp	wait
count	200.00	200.00
mean	121.72	72.46
std	18.72	74.97
min	88.00	2.80
25%	110.40	32.38
50%	119.80	54.75
75%	129.43	87.48
max	221.10	488.70

B2. Raw Histogram and Box Plot - bp and wait

Task: Plot a histogram and box plot of the raw bp column side by side. Repeat for the raw wait column. In a markdown cell, compare the two pairs of plots - which column looks more symmetric, and which shows a longer upper tail? Are outlier dots visible in the box plots?

Solution:

```
[3]: fig,ax=plt.subplots(2,2,figsize=(11,8))
     ax[0,0].hist(df["bp"],bins=20,color="steelblue",edgecolor="white")
     ax[0,0].set_title("Raw - bp - histogram")
     ax[0,0].set_xlabel("bp (mm Hg)")
     ax[0,0].set_ylabel("Count")
     ax[0,1].boxplot(df["bp"])
     ax[0,1].set_title("Raw - bp - box plot")
     ax[0,1].set_ylabel("bp (mm Hg)")
     ax[1,0].hist(df["wait"],bins=20,color="darkorange",edgecolor="white")
     ax[1,0].set_title("Raw - wait - histogram")
     ax[1,0].set_xlabel("wait (minutes)")
     ax[1,0].set_ylabel("Count")
     ax[1,1].boxplot(df["wait"])
     ax[1,1].set_title("Raw - wait - box plot")
     ax[1,1].set_ylabel("wait (minutes)")
     plt.tight_layout()
     plt.show()
```



Answer :

bp looks far more symmetric with an even bell shape, while wait shows a clearly longer upper tail with most values bunched below 100 and a thin spread past 300. Outlier dots are visible above the top whisker in both box plots, but they stand out more sharply for wait.

B3. Skewness of Raw Columns

Task: Compute the skewness of both raw columns using pd.Series.skew(). Record the values in a markdown cell and explain what positive skewness means visually. Which column is more skewed, and does that match what you see in the plots?

Solution:

```
[4]: print("bp skew:",df["bp"].skew())
     print("wait skew:",df["wait"].skew())

     bp skew: 2.2034292021486865
     wait skew: 3.4447160606920275
```

Answer :

bp has a skewness of about 2.20 and wait about 3.44, so wait is more skewed. Positive skewness means a longer right tail, which matches the long right tail seen in the wait histogram compared to bp's near-symmetric bell.

B.2 Min Scaling (x - min)

B4. Apply Min Scaling

Task: Apply min scaling to both columns. Print the min and max of each result and confirm that every minimum is now exactly 0.

Solution:

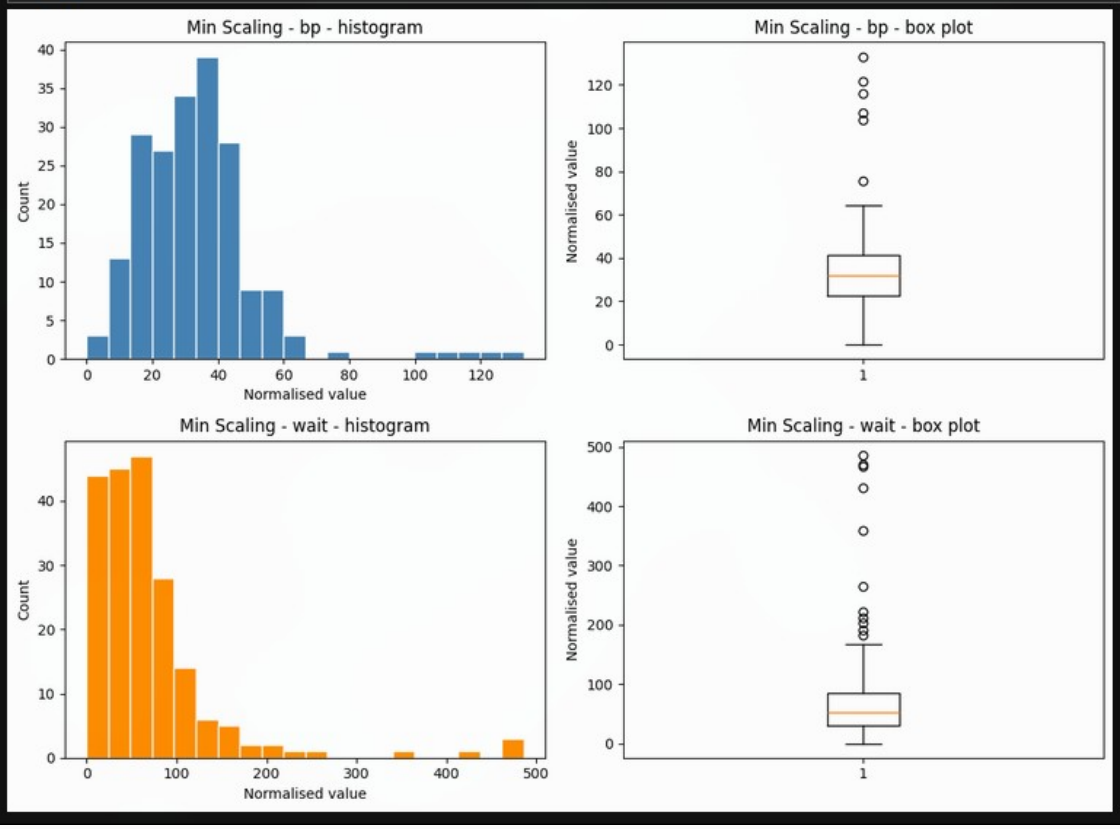
```
[5]: bp_min=df["bp"]-df["bp"].min()
     wait_min=df["wait"]-df["wait"].min()
     print("bp_min range:",bp_min.min(),bp_min.max())
     print("wait_min range:",wait_min.min(),wait_min.max())

     bp_min range: 0.0 133.1
     wait_min range: 0.0 485.9
```

B5. Min Scaling - Histogram and Box Plot

Task: For each column, plot a histogram and box plot side by side (label the technique and column in the title). Write one sentence per column: what changed on the x-axis compared to the raw plots, and what stayed the same?

Solution:

```
[6]: fig,ax=plt.subplots(2,2,figsize=(11,8))
     ax[0,0].hist(bp_min,bins=20,color="steelblue",edgecolor="white")
     ax[0,0].set_title("Min Scaling - bp - histogram")
     ax[0,0].set_xlabel("Normalised value")
     ax[0,0].set_ylabel("Count")
     ax[0,1].boxplot(bp_min)
     ax[0,1].set_title("Min Scaling - bp - box plot")
     ax[0,1].set_ylabel("Normalised value")
     ax[1,0].hist(wait_min,bins=20,color="darkorange",edgecolor="white")
     ax[1,0].set_title("Min Scaling - wait - histogram")
     ax[1,0].set_xlabel("Normalised value")
     ax[1,0].set_ylabel("Count")
     ax[1,1].boxplot(wait_min)
     ax[1,1].set_title("Min Scaling - wait - box plot")
     ax[1,1].set_ylabel("Normalised value")
     plt.tight_layout()
     plt.show()
```



Answer :

For bp the x-axis now starts at 0 and runs to about 133 instead of 88 to 221, and for wait it starts at 0 and runs to about 486 instead of 2.8 to 488.7. Both the bell and skewed shapes stayed exactly the same, only the numbers on the x-axis moved.

B6. Usefulness and Limitation of Min Scaling

Task: In a markdown cell, state one situation where shifting a distribution so its minimum is 0 is useful, and one limitation of this technique.

Solution:

So shifting a distribution so its minimum is 0 is useful when a downstream calculation or plot cannot handle negative numbers, for example if you want to treat the values as physical quantities like a duration or a count where negative values would not make sense, min scaling guarantees a non-negative starting point without touching the shape. the limitation is that it does not control the overall scale at all - the maximum value still depends entirely on the outliers, so two columns shifted this way can end up on very different numeric ranges and still cannot be compared directly or fed into a distance-based model together.

B.3 Max Scaling (x / max)

B7. Apply Max Scaling

Task: Apply max scaling to both columns. Print the min and max of each result. Is the minimum of bp_max the same as the minimum of bp_mm? Why or why not?

Solution:

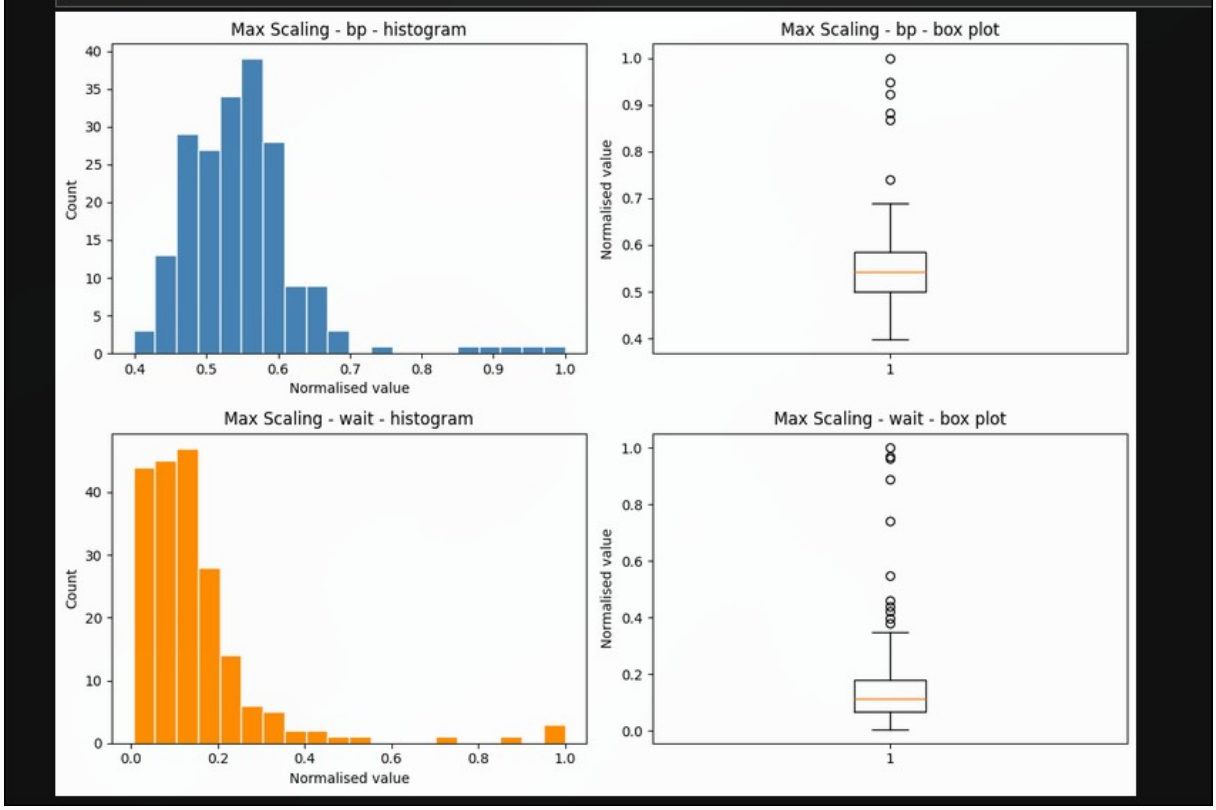
```
[7]: bp_max=df["bp"]/df["bp"].max()
     wait_max=df["wait"]/df["wait"].max()
     print("bp_max range:",bp_max.min(),bp_max.max())
     print("wait_max range:",wait_max.min(),wait_max.max())

     bp_max range: 0.39800995024875624 1.0
     wait_max range: 0.005729486392469818 1.0
```

B8. Max Scaling - Histogram and Box Plot

Task: Plot histogram + box plot for each column after max scaling. In a markdown cell, compare these plots with the min scaling plots from B.2: what is different, and why does the minimum of the max-scaled column not equal 0?

Solution:

```
[8]: fig,ax=plt.subplots(2,2,figsize=(11,8))
     ax[0,0].hist(bp_max,bins=20,color="steelblue",edgecolor="white")
     ax[0,0].set_title("Max Scaling - bp - histogram")
     ax[0,0].set_xlabel("Normalised value")
     ax[0,0].set_ylabel("Count")
     ax[0,1].boxplot(bp_max)
     ax[0,1].set_title("Max Scaling - bp - box plot")
     ax[0,1].set_ylabel("Normalised value")
     ax[1,0].hist(wait_max,bins=20,color="darkorange",edgecolor="white")
     ax[1,0].set_title("Max Scaling - wait - histogram")
     ax[1,0].set_xlabel("Normalised value")
     ax[1,0].set_ylabel("Count")
     ax[1,1].boxplot(wait_max)
     ax[1,1].set_title("Max Scaling - wait - box plot")
     ax[1,1].set_ylabel("Normalised value")
     plt.tight_layout()
     plt.show()
```



Answer :

Compared with min scaling, these histograms are squeezed into a much narrower x-axis window since dividing by the maximum caps every value at 1. The minimum is not 0 here because max scaling only divides by the largest value, it never subtracts the minimum.

B.4 Min-Max Scaling ((x - min) / (max - min))

B9. Apply Min-Max Scaling

Task: Apply min-max scaling to both columns. Confirm that every minimum is 0 and every maximum is 1.

Solution:

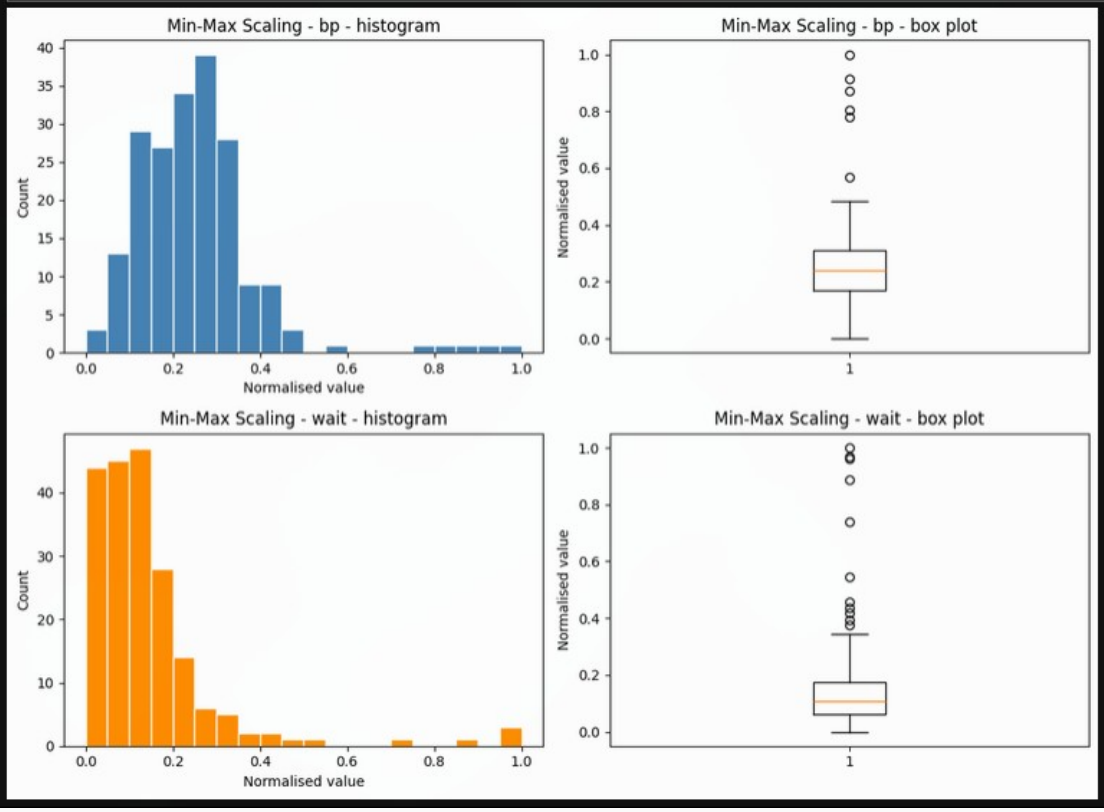
```
[9]: bp_mm=(df["bp"]-df["bp"].min())/(df["bp"].max()-df["bp"].min())
     wait_mm=(df["wait"]-df["wait"].min())/(df["wait"].max()-df["wait"].min())
     print("bp_mm range:",bp_mm.min(),bp_mm.max())
     print("wait_mm range:",wait_mm.min(),wait_mm.max())

     bp_mm range: 0.0 1.0
     wait_mm range: 0.0 1.0
```

B10. Min-Max Scaling - Histogram and Box Plot

Task: Plot histogram + box plot for each column. In a markdown cell, locate the outlier dots on the box plots: do they look more squashed towards the high end compared with the z-score and IQR plots you will draw later?

Solution:

```
[10]: fig,ax=plt.subplots(2,2,figsize=(11,8))
      ax[0,0].hist(bp_mm,bins=20,color="steelblue",edgecolor="white")
      ax[0,0].set_title("Min-Max Scaling - bp - histogram")
      ax[0,0].set_xlabel("Normalised value")
      ax[0,0].set_ylabel("Count")
      ax[0,1].boxplot(bp_mm)
      ax[0,1].set_title("Min-Max Scaling - bp - box plot")
      ax[0,1].set_ylabel("Normalised value")
      ax[1,0].hist(wait_mm,bins=20,color="darkorange",edgecolor="white")
      ax[1,0].set_title("Min-Max Scaling - wait - histogram")
      ax[1,0].set_xlabel("Normalised value")
      ax[1,0].set_ylabel("Count")
      ax[1,1].boxplot(wait_mm)
      ax[1,1].set_title("Min-Max Scaling - wait - box plot")
      ax[1,1].set_ylabel("Normalised value")
      plt.tight_layout()
      plt.show()
```



Answer :

The outlier dots on both box plots sit very close to 1.0, squashed right against the top of the [0,1] range. This looks more compressed toward the high end than the z-score and IQR plots, since one extreme value defines the entire scale.

B11. Why Min-Max Scaling is Sensitive to Outliers

Task: In a markdown cell, explain why min-max scaling is sensitive to outliers. What would happen to all the non-outlier data points if a single value of 10,000 were added to the wait column?

Solution:

So min-max scaling is sensitive to outliers because the entire [0,1] range is defined by
just two numbers, the minimum and the maximum, so a single extreme value completely determines where 1.0 falls. if a value of 10,000 were added to the wait column, the new maximum would jump from about 489 to 10,000, and every existing non-outlier value would be divided by a denominator roughly twenty times larger than before - all of them would collapse into a tiny sliver near 0, destroying the resolution
between typical waiting times that used to be spread out across the full [0,1] range.

B.5 Z-Score Normalisation ((x - mean) / std)

B12. Apply Z-Score Normalisation

Task: Apply z-score normalisation to both columns. Print the mean and standard deviation of each result - they should be approximately 0 and 1. Are they exactly 0 and 1? Why might there be a tiny numerical difference?

Solution:

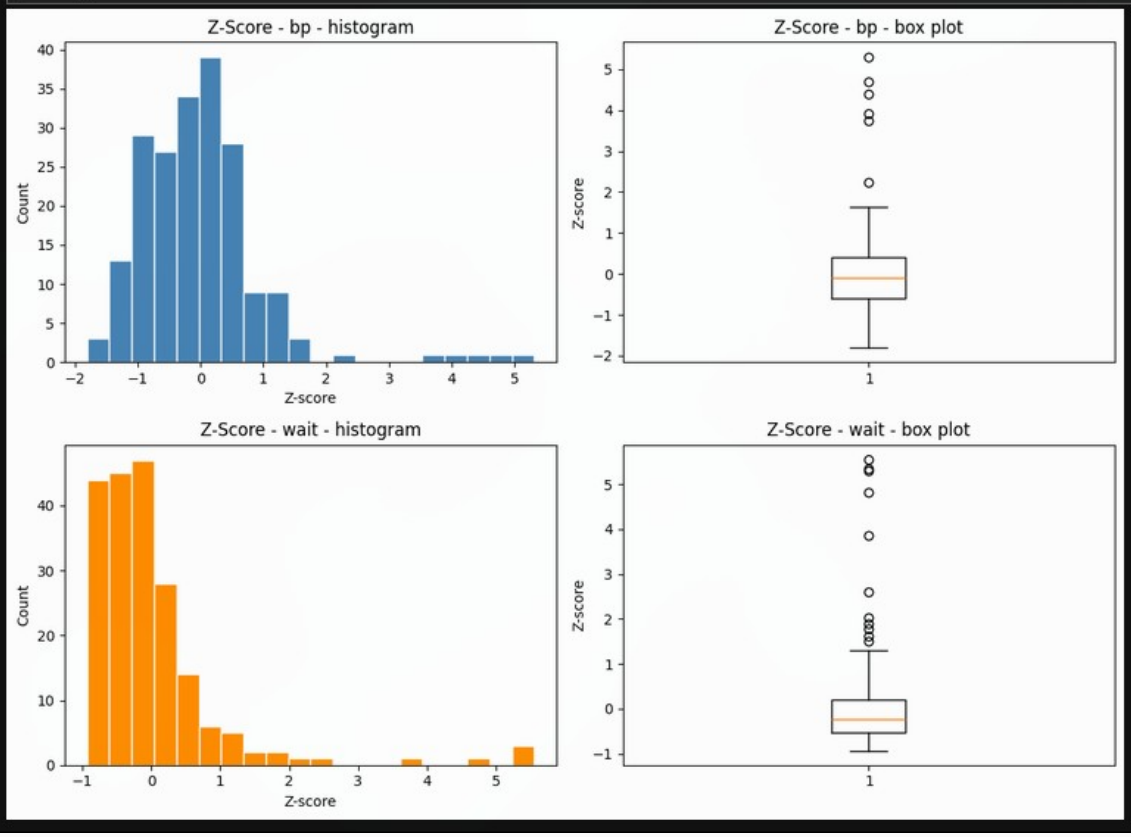
```
[11]: bp_z=(df["bp"]-df["bp"].mean())/df["bp"].std()
      wait_z=(df["wait"]-df["wait"].mean())/df["wait"].std()
      print("bp_z mean and std:",bp_z.mean(),bp_z.std())
      print("wait_z mean and std:",wait_z.mean(),wait_z.std())

      bp_z mean and std: 3.7747582837255325e-17 0.9999999999999999
      wait_z mean and std: 8.881784197001253e-18 1.0
```

B13. Z-Score - Histogram and Box Plot

Task: Plot histogram + box plot for each column. In a markdown cell, identify the z-score value of the outliers from the box plot. Are any beyond +-3? What does a z-score of +4 mean in plain language?

Solution:

[12]:
```
fig,ax=plt.subplots(2,2,figsize=(11,8))
ax[0,0].hist(bp_z,bins=20,color="steelblue",edgecolor="white")
ax[0,0].set_title("Z-Score - bp - histogram")
ax[0,0].set_xlabel("Z-score")
ax[0,0].set_ylabel("Count")
ax[0,1].boxplot(bp_z)
ax[0,1].set_title("Z-Score - bp - box plot")
ax[0,1].set_ylabel("Z-score")
ax[1,0].hist(wait_z,bins=20,color="darkorange",edgecolor="white")
ax[1,0].set_title("Z-Score - wait - histogram")
ax[1,0].set_xlabel("Z-score")
ax[1,0].set_ylabel("Count")
ax[1,1].boxplot(wait_z)
ax[1,1].set_title("Z-Score - wait - box plot")
ax[1,1].set_ylabel("Z-score")
plt.tight_layout()
plt.show()
```



Answer :

The outlier dots on bp reach up to about z=5.3 and on wait up to about z=5.6, both well beyond +-3. A z-score of +4 means that value is 4 standard deviations above the average, an extremely rare reading in a roughly normal distribution.

B14. Compare Z-Score Histogram of bp with Raw

Task: Compare the z-score histogram of bp with its raw histogram from B.1. Write one sentence: how did the shape change, and how did the x-axis scale change?

Solution:

The bell shape of the bp histogram is unchanged after z-scoring, but the x-axis scale changed completely - it used to run from about 88 to 221 in mm Hg, and now it runs from about -1.8 to 5.3 in standard-deviation units centred on 0, so the same bell just slid over so its centre sits at zero instead of at 120.

B.6 IQR / Robust Scaling ((x - median) / IQR)

B15. Apply IQR / Robust Scaling

Task: Apply IQR scaling to both columns. Print the median and IQR of each result - the median of the normalised column should be exactly 0. Verify this.

Solution:

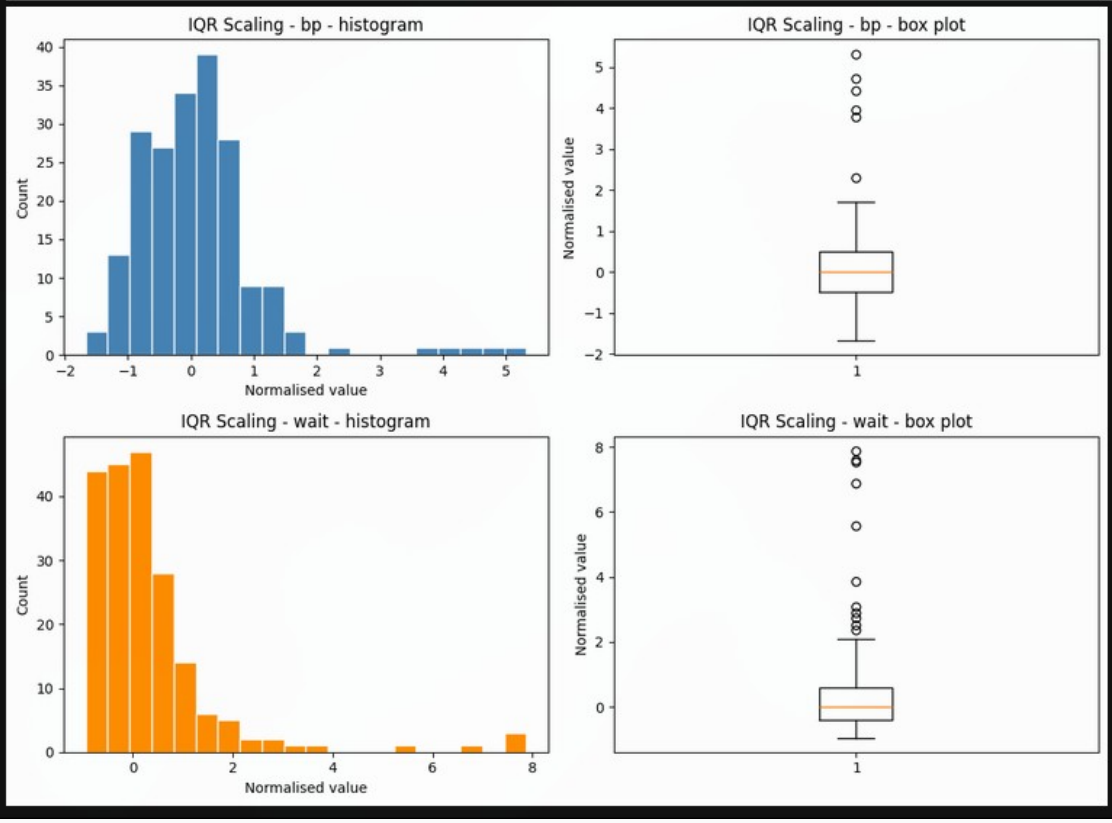
```
[13]: Q1_bp,Q3_bp=df["bp"].quantile(0.25),df["bp"].quantile(0.75)
      Q1_wait,Q3_wait=df["wait"].quantile(0.25),df["wait"].quantile(0.75)
      bp_iqr=(df["bp"]-df["bp"].median())/(Q3_bp-Q1_bp)
      wait_iqr=(df["wait"]-df["wait"].median())/(Q3_wait-Q1_wait)
      print("bp_iqr median:",bp_iqr.median(),"IQR value:",Q3_bp-Q1_bp)
      print("wait_iqr median:",wait_iqr.median(),"IQR value:",Q3_wait-Q1_wait)

      bp_iqr median: -3.7339922820400773e-16 IQR value: 19.025000000000006
      wait_iqr median: 0.0 IQR value: 55.099999999999994
```

B16. IQR Scaling - Histogram and Box Plot

Task: Plot histogram + box plot for each column. In a markdown cell, observe the outlier dots on the box plots: are they farther from the box compared with z-score normalisation? Explain why IQR scaling pushes outliers further out while keeping the main body of the data more compact.

Solution:

```
[14]: fig,ax=plt.subplots(2,2,figsize=(11,8))
      ax[0,0].hist(bp_iqr,bins=20,color="steelblue",edgecolor="white")
      ax[0,0].set_title("IQR Scaling - bp - histogram")
      ax[0,0].set_xlabel("Normalised value")
      ax[0,0].set_ylabel("Count")
      ax[0,1].boxplot(bp_iqr)
      ax[0,1].set_title("IQR Scaling - bp - box plot")
      ax[0,1].set_ylabel("Normalised value")
      ax[1,0].hist(wait_iqr,bins=20,color="darkorange",edgecolor="white")
      ax[1,0].set_title("IQR Scaling - wait - histogram")
      ax[1,0].set_xlabel("Normalised value")
      ax[1,0].set_ylabel("Count")
      ax[1,1].boxplot(wait_iqr)
      ax[1,1].set_title("IQR Scaling - wait - box plot")
      ax[1,1].set_ylabel("Normalised value")
      plt.tight_layout()
      plt.show()
```



Answer :

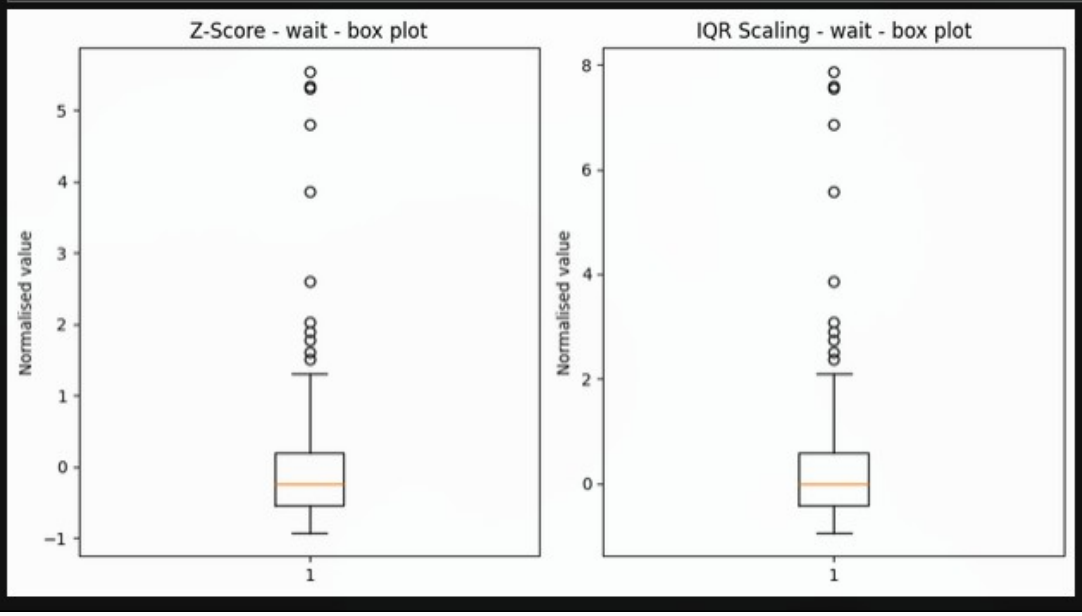
On wait, the IQR-scaled outliers reach out to about 7.9, farther than the z-scaled outliers at about 5.6. IQR scaling divides by the interquartile range, which stays small even with big outliers, so those outliers get pushed out much farther while the main body stays compact.

B17. IQR vs Z-Score Box Plot Comparison for wait

Task: Compare the IQR-scaled box plot of wait with the z-score-scaled box plot of wait side by side. In a markdown cell, state which method you would prefer for a dataset with many extreme outliers and give one reason.

Solution:

```
[15]: fig,ax=plt.subplots(1,2,figsize=(9,5))
      ax[0].boxplot(wait_z)
      ax[0].set_title("Z-Score - wait - box plot")
      ax[0].set_ylabel("Normalised value")
      ax[1].boxplot(wait_iqr)
      ax[1].set_title("IQR Scaling - wait - box plot")
      ax[1].set_ylabel("Normalised value")
      plt.tight_layout()
      plt.show()
```



Answer :

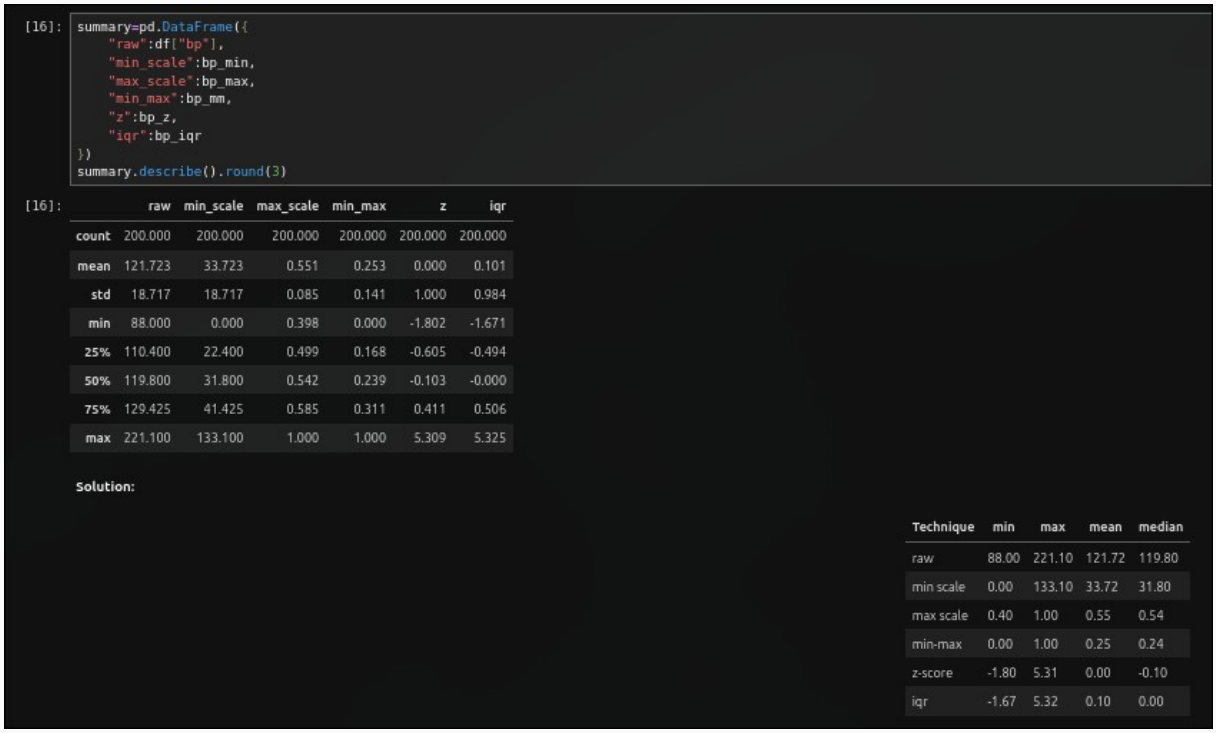
For a dataset with many extreme outliers, IQR/robust scaling is preferred over z-score, since the standard deviation used in z-scoring is itself inflated by the outliers. The median and IQR barely move with extreme values, so the everyday data keeps a more faithful, spread-out shape.

B.7 Summary Comparison

B18. Summary DataFrame of All Five Normalisations of bp

Task: Create a single DataFrame containing the raw column and all five normalised versions of bp. Call describe() on it and print the result. In a markdown cell, fill in the observation table (technique, min, max, mean, median). Which two techniques produce the same shape with different x-axis scales?

Solution:



[16]:
```
summary=pd.DataFrame({
    "raw":df["bp"],
    "min_scale":bp_min,
    "max_scale":bp_max,
    "min_max":bp_mm,
    "z":bp_z,
    "iqr":bp_iqr
})
summary.describe().round(3)
```

[16]:

	raw	min_scale	max_scale	min_max	z	iqr
count	200.000	200.000	200.000	200.000	200.000	200.000
mean	121.723	33.723	0.551	0.253	0.000	0.101
std	18.717	18.717	0.085	0.141	1.000	0.984
min	88.000	0.000	0.398	0.000	-1.802	-1.671
25%	110.400	22.400	0.499	0.168	-0.605	-0.494
50%	119.800	31.800	0.542	0.239	-0.103	-0.000
75%	129.425	41.425	0.585	0.311	0.411	0.506
max	221.100	133.100	1.000	1.000	5.309	5.325

Solution:

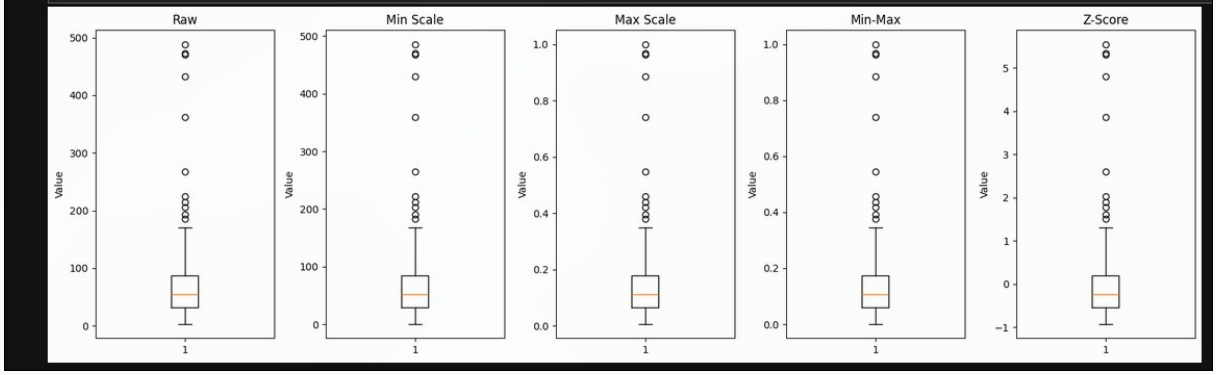
Technique	min	max	mean	median
raw	88.00	221.10	121.72	119.80
min scale	0.00	133.10	33.72	31.80
max scale	0.40	1.00	0.55	0.54
min-max	0.00	1.00	0.25	0.24
z-score	-1.80	5.31	0.00	-0.10
iqr	-1.67	5.32	0.10	0.00

B19. Five Box Plots Side by Side for wait

Task: Draw one figure with five box plots side by side - one per normalisation technique - for the wait column. Write two sentences: which technique most compresses the main body of the data, and which technique makes outliers look the most extreme?

Solution:

```
[17]: fig,ax=plt.subplots(1,5,figsize=(16,5))
      techniques=[("Raw",df["wait"]),("Min Scale",wait_min),("Max Scale",wait_max),("Min-Max",wait_mm),("Z-Score",wait_z)]
      for a,(name,data) in zip(ax,techniques):
          a.boxplot(data)
          a.set_title(name)
          a.set_ylabel("Value")
      plt.tight_layout()
      plt.show()
```



Answer :

Min-max scaling most compresses the main body of the data, squeezing almost every ordinary point into a thin band near the bottom once the outlier defines the top. Z-score normalisation makes the outliers look the most extreme, stretching out to roughly +5 away from the tightly clustered main body.

B20. Choosing a Technique for an Outlier-Sensitive Model

Task: In a final markdown cell, write a short paragraph (4-6 sentences) answering: for a machine learning model that is sensitive to the scale of features (e.g., K-NN), which normalisation technique would you choose for a dataset that contains outliers, and why? Mention at least two techniques in your comparison.

Solution:

For a distance-based model like K-NN on data that contains outliers, IQR / robust scaling is the better default choice over min-max scaling. min-max scaling anchors the entire [0,1] range to the single smallest and largest values, so one extreme outlier can silently compress every ordinary point into a narrow band, making K-NN's distance calculations lose most of their ability to discriminate between typical cases. z-score is a reasonable middle ground, since it centres the data around 0, but its scale still depends on the standard deviation which outliers inflate. robust scaling uses the median and interquartile range instead, both of which barely move when a few extreme points are present, so the bulk of the data keeps a stable, well-spread-out scale and the outliers, rather than distorting everyone else, are simply the ones that end up flagged with unusually large normalised values. that combination of stability for normal points and honesty about which points are unusual is exactly what a distance-based model needs.

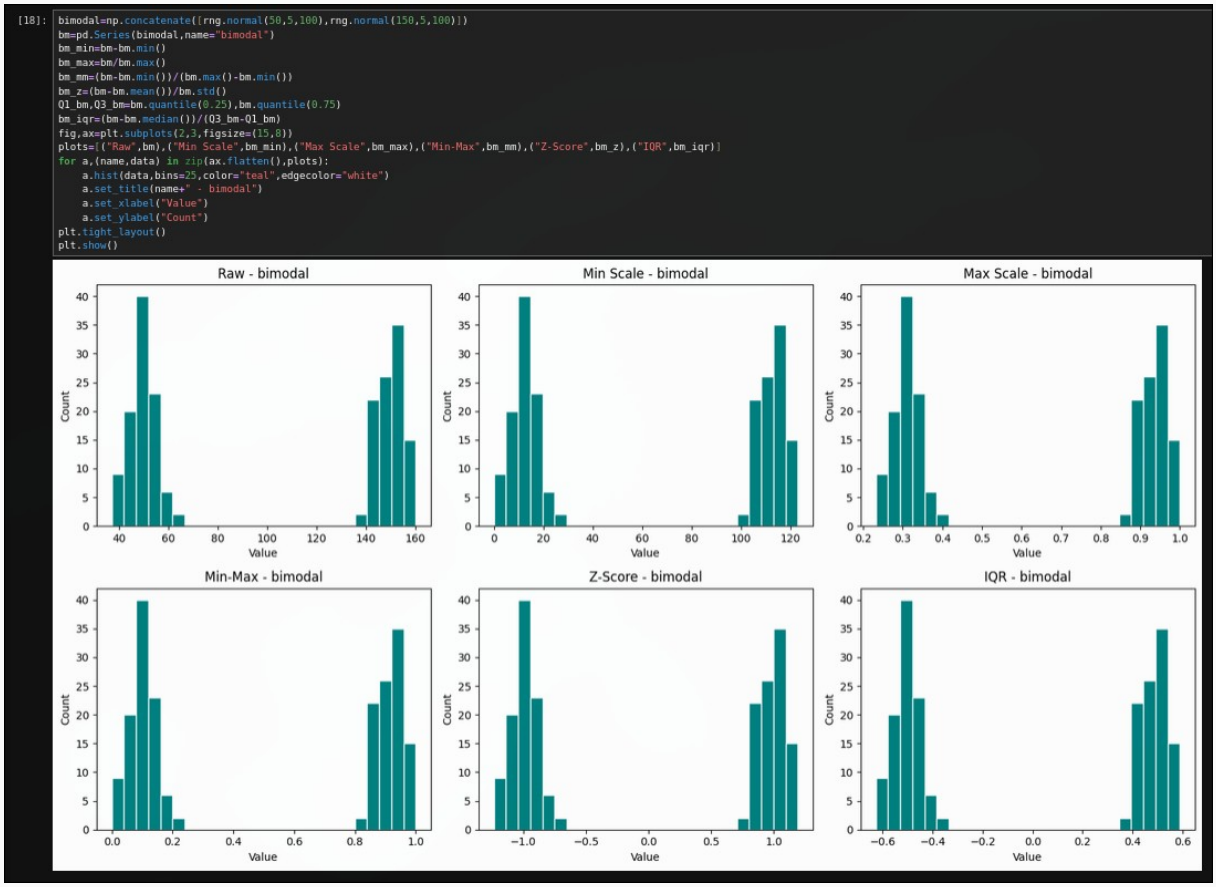
B.8 Take-Home

B21. Bimodal Column - All Five Normalisations

Task: Generate a new bimodal column and apply all five normalisations to it. For each, draw a histogram. Which normalisation preserves the bimodal shape most clearly? Which technique would be misleading if you assumed the output was always unimodal?

Solution:

```
[18]: bimodal=np.concatenate([rng.normal(50,5,100),rng.normal(150,5,100)])
      bm=pd.Series(bimodal,name="bimodal")
      bm_min=bm-bm.min()
      bm_max=bm/bm.max()
      bm_mm=(bm-bm.min())/(bm.max()-bm.min())
      bm_z=(bm-bm.mean())/bm.std()
      Q1_bm,Q3_bm=bm.quantile(0.25),bm.quantile(0.75)
      bm_iqr=(bm-bm.median())/(Q3_bm-Q1_bm)
      fig,ax=plt.subplots(2,3,figsize=(15,8))
      plots=[("Raw",bm),("Min Scale",bm_min),("Max Scale",bm_max),("Min-Max",bm_mm),("Z-Score",bm_z),("IQR",bm_iqr)]
      for a,(name,data) in zip(ax.flatten(),plots):
          a.hist(data,bins=25,color="teal",edgecolor="white")
          a.set_title(name+" - bimodal")
          a.set_xlabel("Value")
          a.set_ylabel("Count")
      plt.tight_layout()
      plt.show()
```



Answer :

All five normalisations - min scale, max scale, min-max, z-score and IQR - preserve the two separate humps just as clearly as the raw histogram, since each is a simple linear shift or stretch of the x-axis. None of them would be misleading about the bimodal shape; only a non-linear transform like log could hide it.

B22. Custom min_max_scale and z_score_scale Functions

Task: Without using any pandas or NumPy normalisation helper, write your own functions min_max_scale(arr) and z_score_scale(arr) using only basic Python arithmetic. Verify that their output matches your earlier pandas results for the bp column (hint: np.allclose is useful here).

Solution:

[19]:
```
def min_max_scale(arr):
    lo=arr[0]
    hi=arr[0]
    for x in arr:
        if x<lo:
            lo=x
        if x>hi:
            hi=x
    result=[]
    for x in arr:
        result.append((x-lo)/(hi-lo))
    return result

def z_score_scale(arr):
    n=len(arr)
    total=0
    for x in arr:
        total+=x
    mean=total/n
    sq=0
    for x in arr:
        sq+=(x-mean)**2
    std=(sq/(n-1))**0.5
    result=[]
    for x in arr:
        result.append((x-mean)/std)
    return result

bp_list=df["bp"].tolist()
my_mm=min_max_scale(bp_list)
my_z=z_score_scale(bp_list)
pandas_mm=((df["bp"]-df["bp"].min())/(df["bp"].max()-df["bp"].min())).tolist()
pandas_z=((df["bp"]-df["bp"].mean())/df["bp"].std()).tolist()
print("min-max match:",np.allclose(my_mm,pandas_mm))
print("z-score match:",np.allclose(my_z,pandas_z))
```

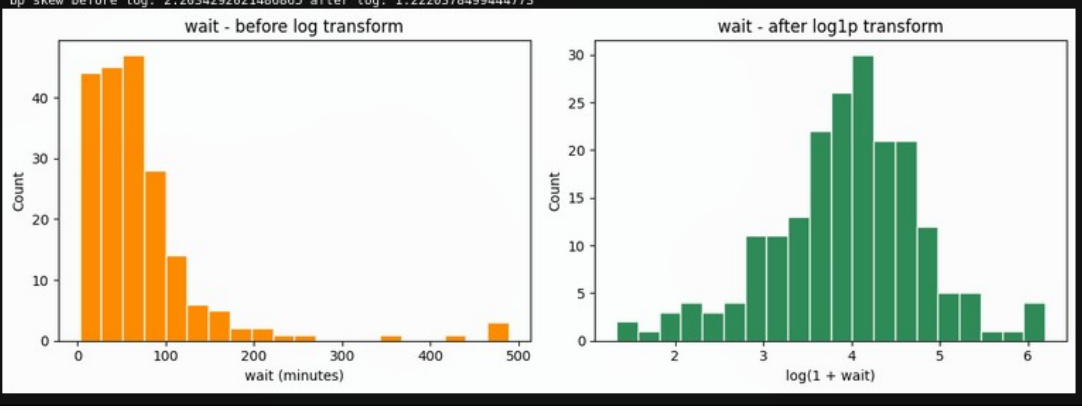
```
min-max match: True
z-score match: True
```

B23. Explore Log Transformation

Task: Explore log transformation on the right-skewed columns and compare the before and after histograms.

Solution:

```
[20]: log_wait=np.log1p(df["wait"])
      log_bp=np.log1p(df["bp"])
      print("wait skew before log:",df["wait"].skew(),"after log:",log_wait.skew())
      print("bp skew before log:",df["bp"].skew(),"after log:",log_bp.skew())
      fig,ax=plt.subplots(1,2,figsize=(11,4))
      ax[0].hist(df["wait"],bins=20,color="darkorange",edgecolor="white")
      ax[0].set_title("wait - before log transform")
      ax[0].set_xlabel("wait (minutes)")
      ax[0].set_ylabel("Count")
      ax[1].hist(log_wait,bins=20,color="seagreen",edgecolor="white")
      ax[1].set_title("wait - after log1p transform")
      ax[1].set_xlabel("log(1 + wait)")
      ax[1].set_ylabel("Count")
      plt.tight_layout()
      plt.show()

      wait skew before log: 3.4447160606920275 after log: -0.3376416945024374
      bp skew before log: 2.2034292021486865 after log: 1.2220378499444773
```



Answer :

Applying log1p to wait drops its skewness from about 3.44 to about -0.34, turning the long right-tailed shape into something close to symmetric. The same transform on bp only drops skew from 2.20 to 1.22, a smaller improvement since bp was not as extremely skewed to begin with.